

Software Design Document

Financial System Monitoring Investments - Software Specification Document

1. Purpose

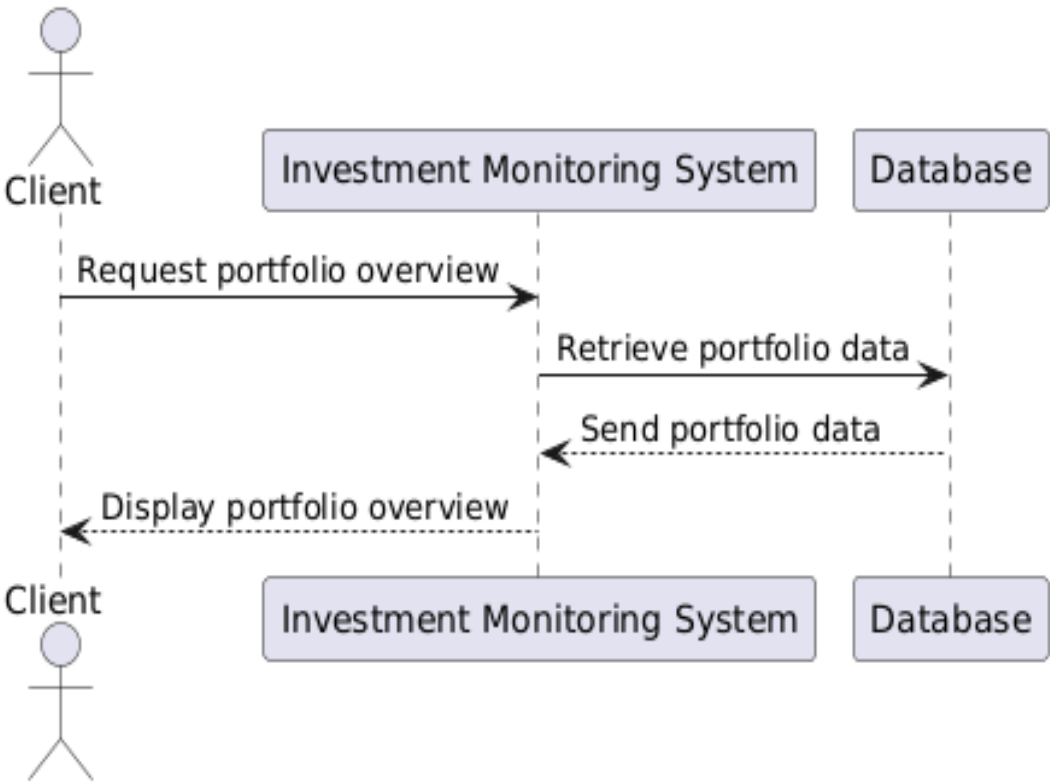
The objective of this software system is to facilitate continuous monitoring and management of investments for our clients. The system is mission-critical, requiring 100% availability and robust security to ensure data privacy and integrity alignment with client priorities.

2. Sequence Diagrams

For each of the identified use cases, corresponding sequence diagrams are prepared as follows:

Use Case 1: UC01 - Monitor Investment Portfolio

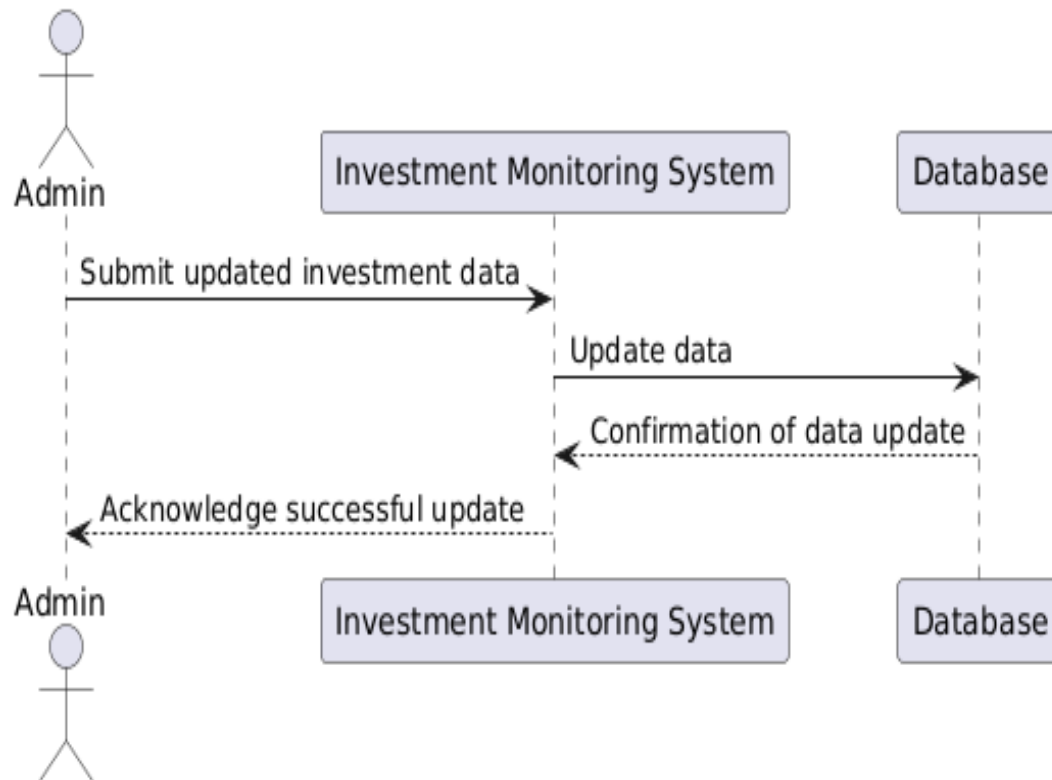
``plantuml



...

Use Case 2: UC02 - Update Investment Data

``plantuml



...

3. Major Design Decisions

Modularization Criteria:

The system will be modularized around the main functionalities: Data Retrieval, Data Processing, and User Interface. Each module will have distinct responsibilities to minimize interdependencies, facilitating better cohesion and lower coupling.

Non-Functional Requirements:

1. **Security**: As client data privacy is paramount, security measures including encryption and stringent authentication will be integral.
2. **Reliability**: The system's design will focus on maintaining 100% availability through redundant systems and auto-recovery solutions.

4. Architecture

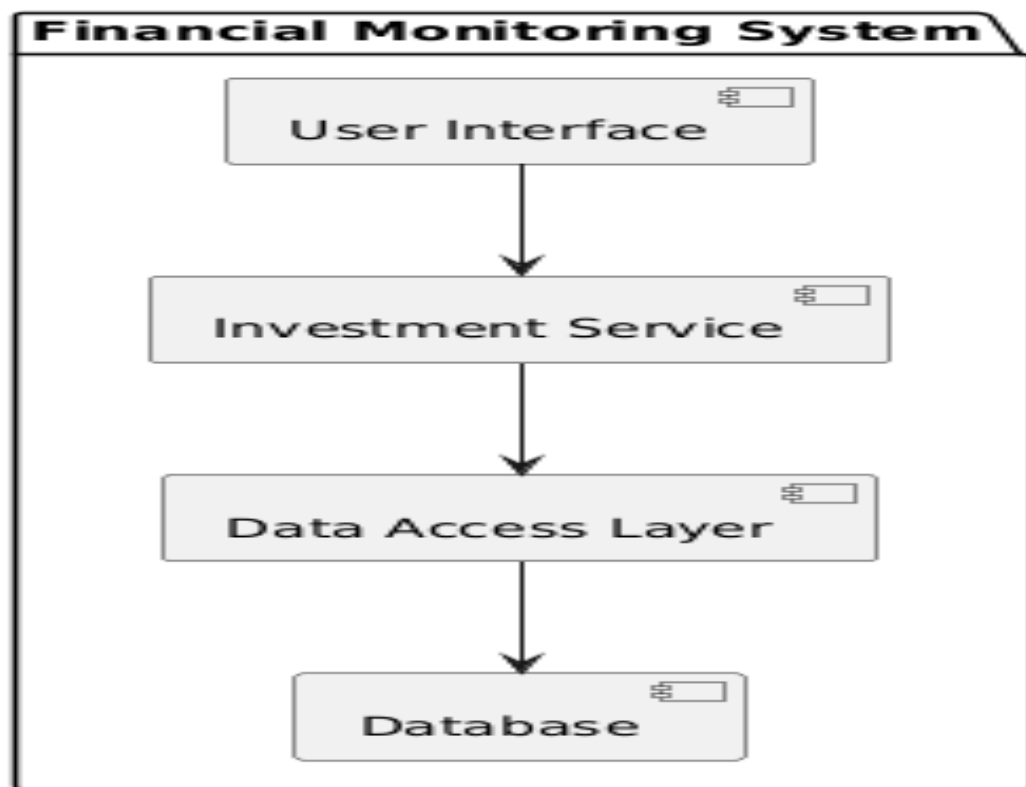
Architectural Patterns:

The system adopts a microservices architecture. This approach aligns with our need to scale individual components independently and maintain continuous availability. Each service will handle a specific aspect of the investment monitoring process, offering scalability and fault isolation.

Additionally, a layered architecture will be applied, segmenting responsibilities into distinct layers (UI, Business Logic, and Data Access), promoting a clean separation of concerns and enhanced manageability.

Component Decomposition:

```
``plantuml
```



```
...
```

- **User Interface**: Manages interactions between the user and the system, providing a dashboard for portfolio monitoring.

- **Investment Service**: Contains the business logic for processing investment data and invoking necessary computations.
- **Data Access Layer**: Handles all database interactions, ensuring data integrity and secure storage.
- **Database**: Stores all investment and user data securely.

Exposed Interfaces:

- **Investment Service Interface**:
 - `getPortfolioData()`: Retrieves investment portfolio data.
 - `updateInvestmentDetails()`: Updates investment details in the database.
- **Data Access Layer Interface**:
 - `queryDatabase()`: Executes a query against the database.
 - `updateDatabaseEntry()`: Performs an update operation on the database.

5. Use of Design Patterns

- **Singleton Pattern** for `Database Connection`: Ensures a single database connection instance, reducing overhead and maintaining consistency.
- **Observer Pattern** for `Portfolio Updates`: Notify users of updates to their monitored portfolios in real-time.

6. Detailed Class Diagrams

User Interface Component

```
``plantuml
```



UserInterface

- displayPortfolio()
- initiateUpdate()

...

Investment Service Component

``plantuml



InvestmentService

- getPortfolioData()
- updateInvestmentDetails()

...

Data Access Layer Component

``plantuml



DataAccessLayer

- queryDatabase()
- updateDatabaseEntry()

...

7. Test Driven Development

Test Case 1

1. **Test ID**: TC01
2. **Category**: Portfolio Data Retrieval
3. **Initial Condition**: System is operational and database is accessible.
4. **Procedure**:
 - Initiate a request for portfolio data from the client interface.
 - System processes the request and queries the database.
5. **Expected Outcome**: Portfolio data is correctly displayed to the client.

Test Case 2

1. **Test ID**: TC02
2. **Category**: Data Update Validation
3. **Initial Condition**: Admin is logged in and has access rights.
4. **Procedure**:
 - Admin submits updated investment data through the system interface.
 - System processes and updates the data in the database.
5. **Expected Outcome**: Confirmation message of successful update is displayed to Admin.